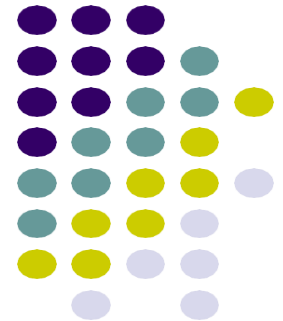


Digital Image Processing (CS/ECE 545)

Lecture 2: Histograms and Point Operations (Part 1)

Prof Emmanuel Agu

*Computer Science Dept.
Worcester Polytechnic Institute (WPI)*





Histograms

- Histograms plots how many times (frequency) each intensity value in image occurs
- Example:
 - Image (left) has 256 distinct gray levels (8 bits)
 - Histogram (right) shows frequency (how many times) each gray level occurs



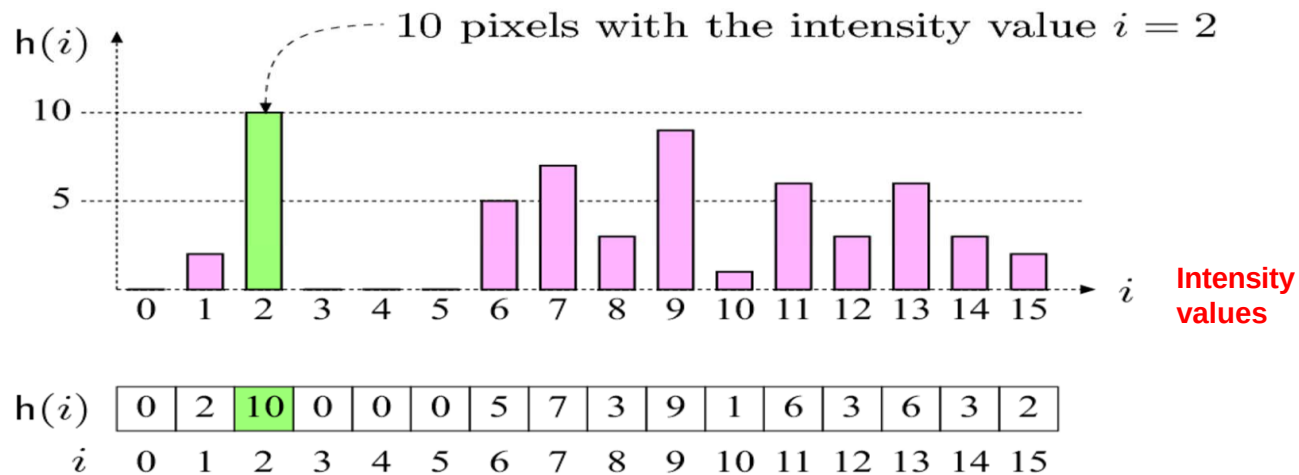


Histograms

- Many cameras display real time histograms of scene
- Helps avoid taking over-exposed pictures
- Also easier to detect types of processing previously applied to image



Histograms

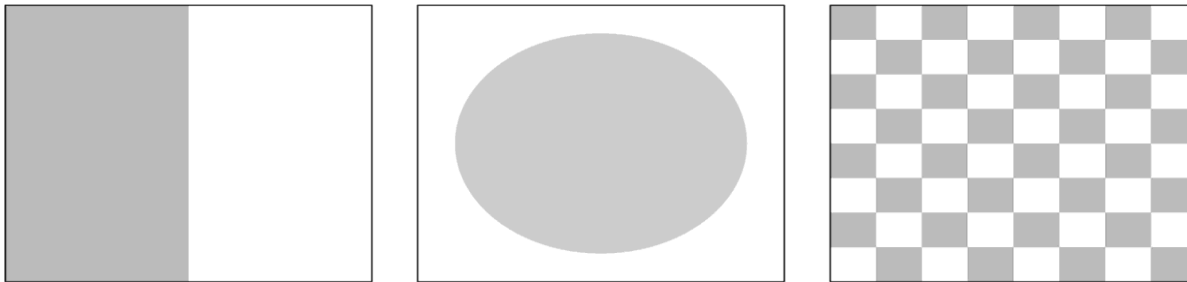


- E.g. $K = 16$, 10 pixels have intensity value = 2
- Histograms: only statistical information
- No indication of **location** of pixels



Histograms

- Different images can have **same** histogram
- 3 images below have same histogram



- Half of pixels are gray, half are white
 - Same histogram = same statistics
 - Distribution of intensities could be different
- Can we reconstruct image from histogram? No!

Histograms



- So, a histogram for a grayscale image with intensity values in range

$$I(u, v) \in [0, K - 1]$$

would contain exactly K entries

- E.g. 8-bit grayscale image, $K = 2^8 = 256$
- Each histogram entry is defined as:

$h(i)$ = number of pixels with intensity i for all $0 \leq i < K$.

- E.g: $h(255)$ = number of pixels with intensity = 255
- Formal definition

$$h(i) = \text{card}\{(u, v) \mid I(u, v) = i\}$$

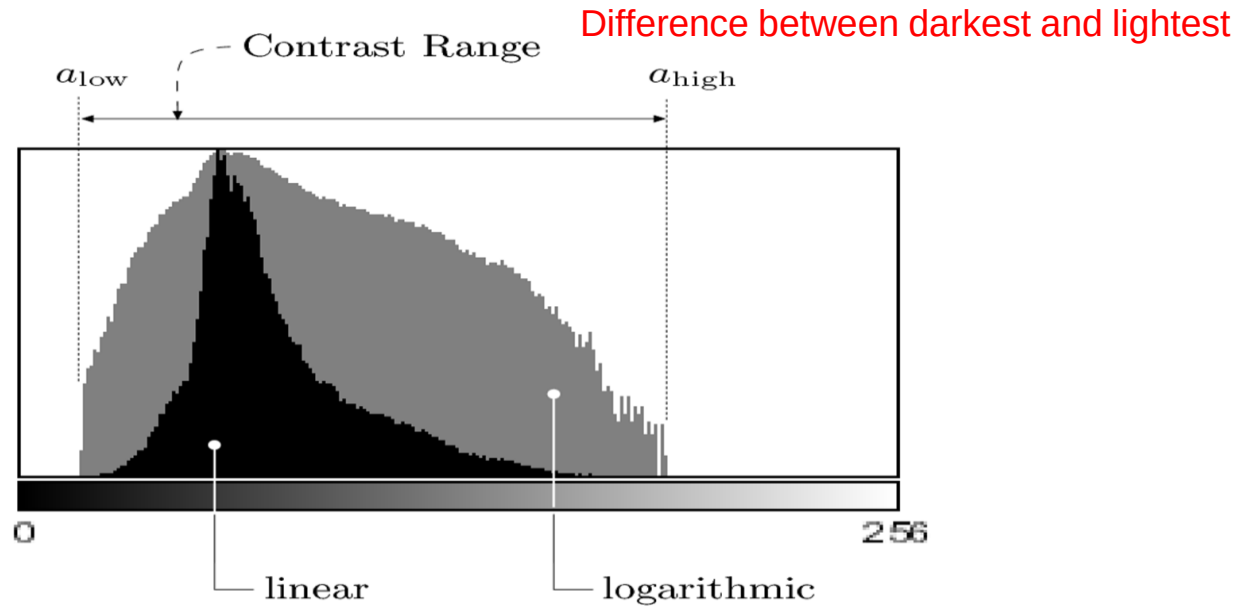
Number (size of set) of pixels

such that



Interpreting Histograms

- Log scale makes low values more visible





Histograms

- Histograms help detect image acquisition issues
- Problems with image can be identified on histogram
 - Over and under exposure
 - Brightness
 - Contrast
 - Dynamic Range
- Point operations can be used to alter histogram. E.g
 - Addition
 - Multiplication
 - Exp and Log
 - Intensity Windowing (Contrast Modification)

Histograms



```
from skimage import data
import matplotlib.pyplot as plt
```

```
# Load built-in grayscale image
I = data.camera()
```

```
# Create a figure with two subplots: one for the image, one for the histogram
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
```

```
# Show the image
ax[0].imshow(I, cmap='gray')
ax[0].set_title('Grayscale Image')
ax[0].axis('off') # Hide axis ticks
```

```
# Show the histogram
ax[1].hist(I.ravel(), bins=256, range=[0, 256])
ax[1].set_title('Histogram')
ax[1].set_xlabel('Pixel Intensity')
ax[1].set_ylabel('Frequency')
```

```
# Display both plots
plt.tight_layout()
plt.show()
```



Image Brightness

- Brightness of a grayscale image is the **average intensity** of all pixels in image

$$B(I) = \frac{1}{wh} \sum_{v=1}^h \sum_{u=1}^w I(u, v)$$

2. Divide by total number of pixels

1. Sum up all pixel intensities



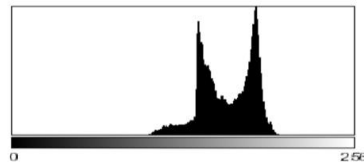
Image Contrast

- The contrast of a grayscale image indicates how easily objects in the image can be distinguished
- **High contrast image:** many distinct intensity values
- **Low contrast:** image uses few intensity values

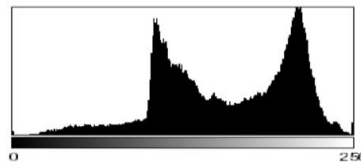


Histograms and Contrast

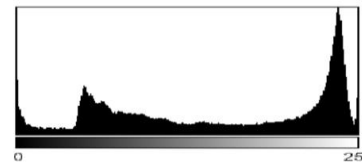
Good Contrast? Widely spread intensity values
+ large difference between min and max intensity values



(a)
Low contrast



(b)
Normal contrast



(c)
High contrast

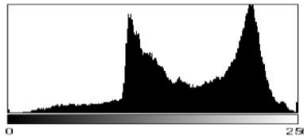
Image

Histogram



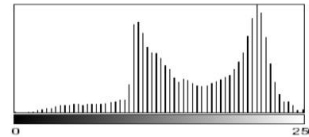
Histograms and Dynamic Range

- **Dynamic Range:** Number of distinct pixels in image



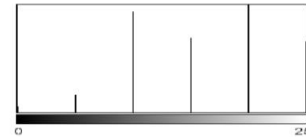
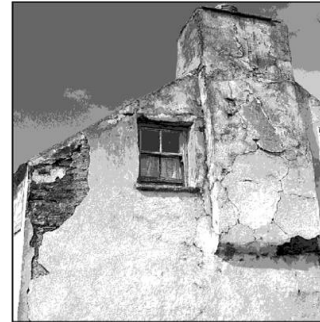
(a)

High Dynamic Range



(b)

**Low Dynamic Range
(64 intensities)**



(c)

**Extremely low
Dynamic Range
(6 intensity values)**



High Dynamic Range Imaging

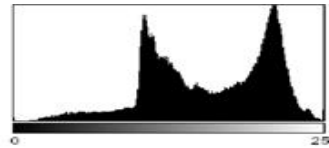
- **High dynamic range** means very bright and very dark parts in a single image (many distinct values)
- Dynamic range in photographed scene may exceed number of available bits to represent pixels
- Solution:
 - Capture multiple images at different exposures
 - Combine them using image processing



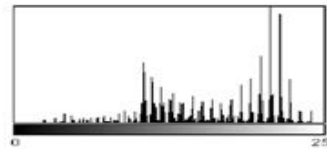
Image Defects: Effect of Image Compression

- Histograms show impact of image compression
- Example: in GIF compression, dynamic range is reduced to only few intensities (quantization)

Original Image



(a) Original Histogram



(b) Histogram after GIF conversion

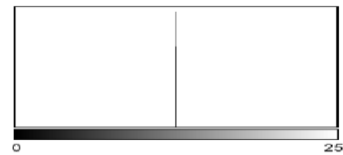


Effect of Image Compression

- Example: Effect of JPEG compression on line graphics
- JPEG compression designed for color images



(a)

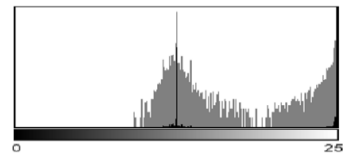


(b)

Original histogram has only 2 intensities (gray and white)



(c)



(d)

JPEG image appears dirty, fuzzy and blurred

Its Histogram contains gray values not in original

Color Image Histograms



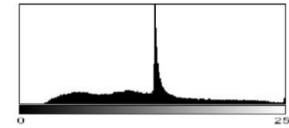
Two types:

1. **Intensity histogram:**

- Convert color image to gray scale
- Display histogram of gray scale



(a)



(b) h_{Lum}



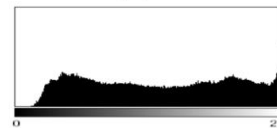
(c) R



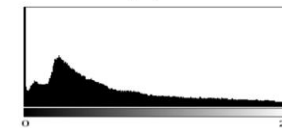
(d) G



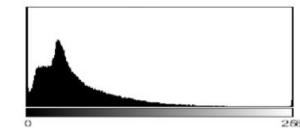
(e) B



(f) h_R



(g) h_G



(h) h_B

2. **Individual Color Channel Histograms:** 3 histograms (R,G,B)



Color Image Histograms

- Histograms provide useful information about lighting, contrast, dynamic range and saturation effects:
 - **Lighting:** Is the image mostly dark, bright, or balanced?
 - **Contrast:** Are there both dark and bright tones, or is everything in the middle?
 - **Dynamic range:** Are values spread across the full 0–255 range, or squished into a small range?
 - **Saturation effects:** If one channel is dominant, the image might look strongly tinted or oversaturated.
- No information about the actual color distribution!
- Images with totally different RGB colors can have same R, G and B histograms



Cumulative Histogram

- Useful for certain operations (e.g. histogram equalization) later
- Analogous to the **Cumulative Density Function (CDF)**

- Definition:

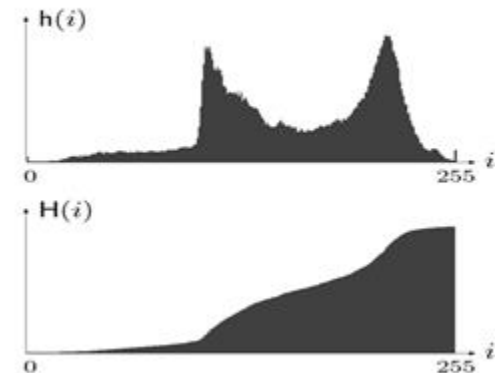
$$H(i) = \sum_{j=0}^i h(j) \quad \text{for } 0 \leq i < K$$

- Recursive definition

$$H(i) = \begin{cases} h(0) & \text{for } i = 0 \\ H(i-1) + h(i) & \text{for } 0 < i < K \end{cases}$$

- Monotonically increasing

$$H(K-1) = \sum_{j=0}^{K-1} h(j)$$





Point Operations

- Point operations changes a pixel's intensity value according to some function (don't care about pixel's neighbor)

$$a' \leftarrow f(a)$$
$$I'(u, v) \leftarrow f(I(u, v))$$

- Also called a **homogeneous operation**
- New pixel intensity **depends on**
 - Pixel's previous intensity $I(u, v)$
 - Mapping function $f()$
- **Does not depend on**
 - Pixel's location (u, v)
 - Intensities of neighboring pixels



Some Homogeneous Point Operations

- Addition (Changes brightness)

$$f(p) = p + k$$

E.g. $f_{\text{bright}}(p) = p + 10$

- Multiplication (Stretches/shrinks image contrast range)

$$f(p) = k \times p$$

E.g. $f_{\text{contrast}}(p) = p \times 1.5$

- Real-valued functions $\exp(x)$, $\log(x)$, $(1/x)$, x^k , etc.

- Quantizing pixel values
- Global thresholding
- Gamma correction



Point Operation Pseudocode

- **Input:** Image with pixel intensities $I(u,v)$ defined on $[1 .. w]$ x $[1 .. H]$
- **Output:** Image with pixel intensities $I'(u,v)$

for $v = 1 .. h$

for $u = 1 .. w$

 set $I(u, v) = f(I(u,v))$



Non-Homogeneous Point Operation

- New pixel value depends on:
 - Old value + **pixel's location (u,v)**

$$\begin{aligned}a' &\leftarrow g(a, u, v) \\ I'(u, v) &\leftarrow g(I(u, v), u, v)\end{aligned}$$



Clamping

- Deals with pixel values outside displayable range
 - If ($a > 255$) $a = 255$;
 - If ($a < 0$) $a = 0$;
- Function below will **clamp** (force) all values to fall within range $[a, b]$

$$f(p) = \begin{cases} a & \text{if } p < a \\ p & \text{if } a \leq p \leq b \\ b & \text{if } p > b \end{cases}$$



Clamping

```
import matplotlib.pyplot as plt
import numpy as np
from skimage import data
```

```
# Step 1: Load the built-in 'camera' image
I = data.camera()
```

```
# Step 2: Display the original image (equivalent to imshow)
plt.imshow(I, cmap='gray')
plt.title("Original Image")
plt.axis('off') # Hide axes for better visualization
plt.show()
```

```
# Step 3: Clip the pixel values (equivalent to the for-loops in MATLAB)
I = np.clip(I, 0, 255)
```

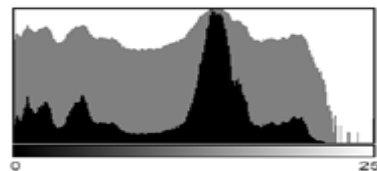
```
# Step 4: Display the processed image (equivalent to second imshow)
plt.imshow(I, cmap='gray')
plt.title("Processed Image")
plt.axis('off') # Hide axes
plt.show()
```



Inverting Images

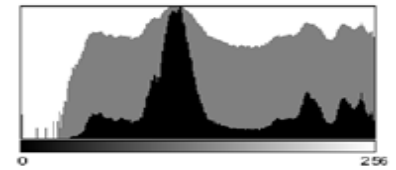
$$f_{\text{invert}}(a) = -a + a_{\text{max}} = a_{\text{max}} - a$$

- 2 steps
 1. Multiple intensity by -1
 2. Add constant (e.g. a_{max}) to put result in range $[0, a_{\text{max}}]$
- Implemented as ImageJ method `invert( )`



(a)

Original



(c)

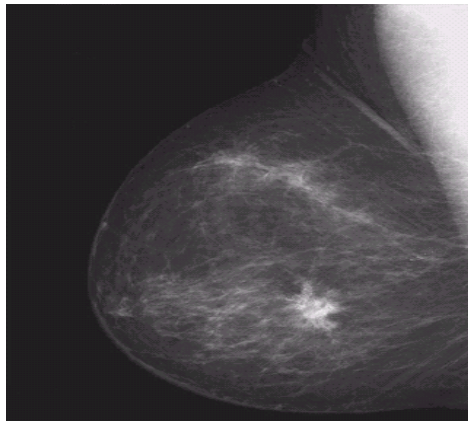
Inverted Image



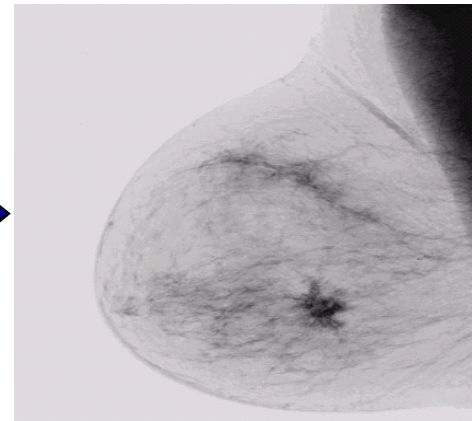
Image Negatives (Inverted Images)

- Image negatives useful for enhancing white or grey detail embedded in dark regions of an image
 - Note how much clearer the tissue is in the negative image of the mammogram below

Original Image



$$s = 1.0 - r$$



Negative Image



Image Negatives (Inverted Images)

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data

# Load a sample grayscale image
I = data.camera()

# Image Negative
negative_image = 255 - I

# Display original and negative image
plt.figure(figsize=(10,5))

plt.subplot(1, 2, 1)
plt.imshow(I, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(negative_image, cmap='gray')
plt.title('Negative Image')
plt.axis('off')

plt.show()
```



Thresholding

- Input values below **threshold** a_{th} set to a_0
- Input values above **threshold** a_{th} set to a_1

$$f_{\text{threshold}}(a) = \begin{cases} a_0 & \text{for } a < a_{th} \\ a_1 & \text{for } a \geq a_{th} \end{cases}$$

- Converts grayscale image to binary image (binarization) if
 - $a_0 = 0$
 - $a_1 = 1$



Thresholding

```
import matplotlib.pyplot as plt
import numpy as np
from skimage import data
```

Step 1: Load the built-in 'camera' image

```
I = data.camera() # This loads the 'camera' image as a NumPy array
```

Step 2: Display the original image

```
plt.imshow(I, cmap='gray')
plt.title("Original Image")
plt.axis('off') # Hide axes for better visualization
plt.show()
```

Step 3: Apply thresholding

```
I[I > 127] = 255 # Set pixels greater than 127 to 255
I[I <= 127] = 0  # Set pixels less than or equal to 127 to 0
```

Step 4: Display the processed image

```
plt.imshow(I, cmap='gray')
plt.title("Processed Image")
plt.axis('off') # Hide axes for better visualization
plt.show()
```

Thresholding Example



Original Image

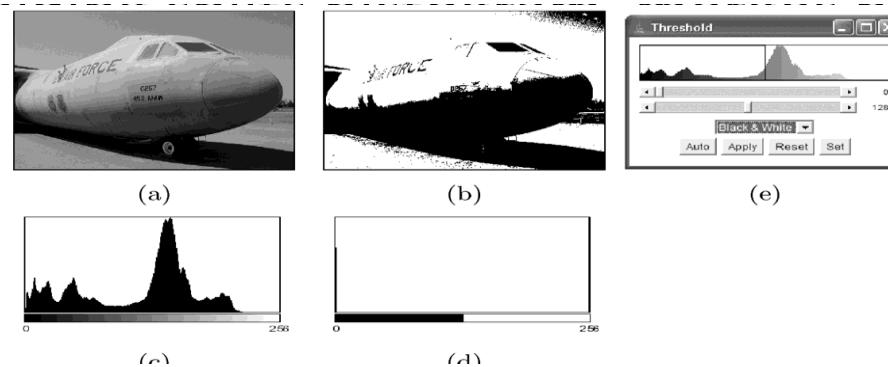


Thresholded Image

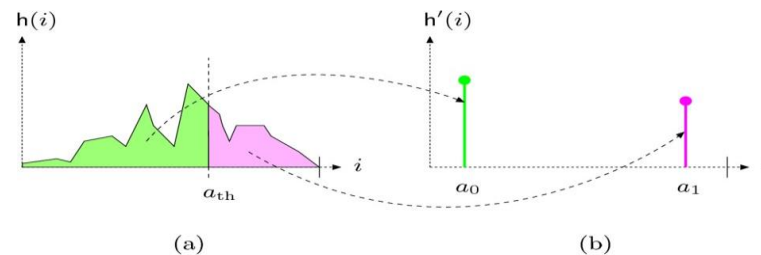


Thresholding and Histograms

- Example with $a_{th} = 128$



- Thresholding splits histogram, merges halves into a_0 a_1

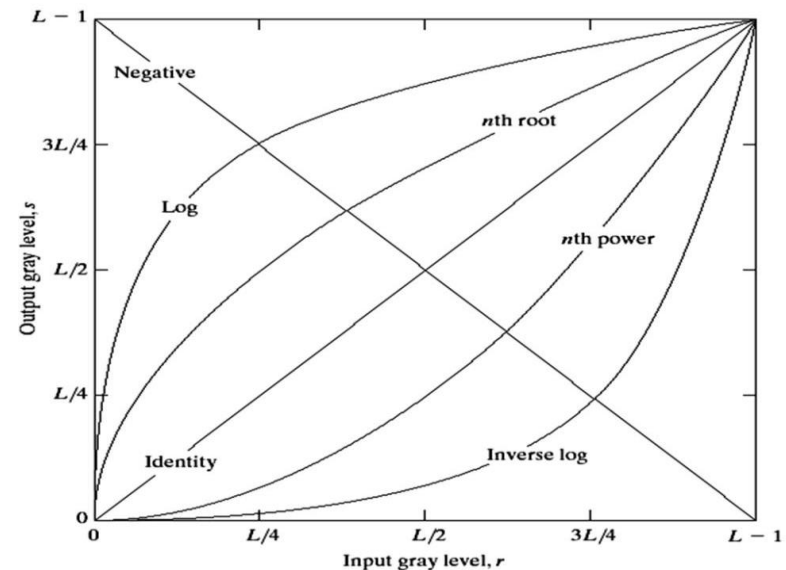




Basic Grey Level Transformations

- 3 most common gray level transformation:

- Linear
 - Negative/Identity
- Logarithmic
 - Log/Inverse log
- Power law
 - n^{th} power/ n^{th} root





Logarithmic Transformations

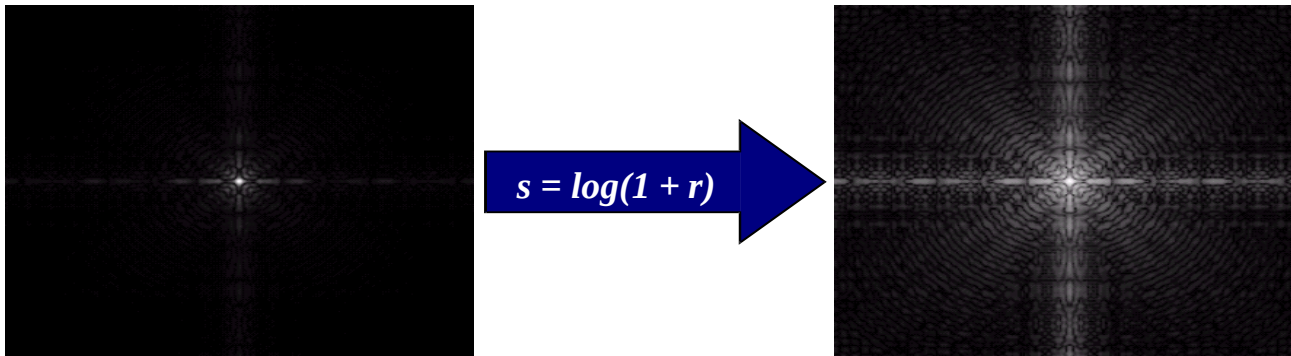
- Maps narrow range of input levels => wider range of output values

- Inverse log transformation does opposite transformation

- The general form of the log transformation is

New pixel value $\longrightarrow s = c * \log(1 + r)$ $\longleftarrow$ Old pixel value

- Log transformation of Fourier transform shows more detail





Power Law Transformations

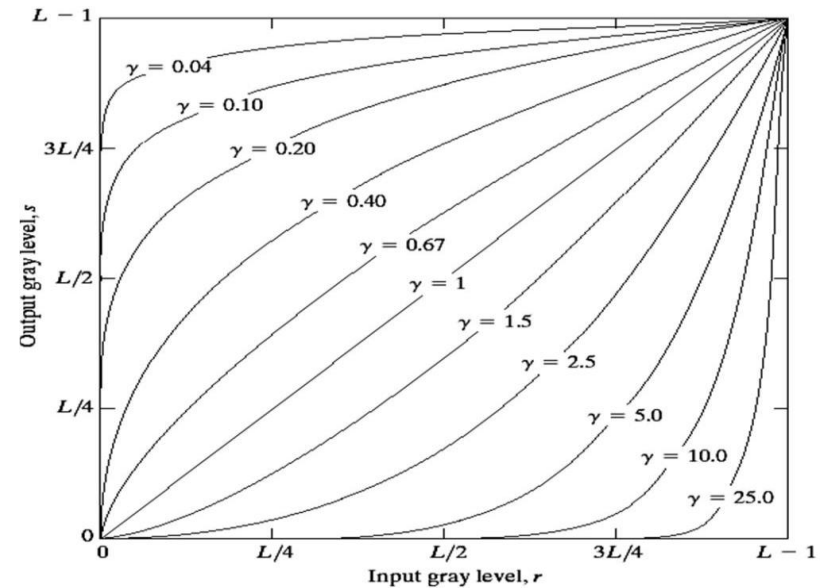
- Power law transformations have the form

$$s = c * r^\gamma$$

Annotations for the equation $s = c * r^\gamma$:

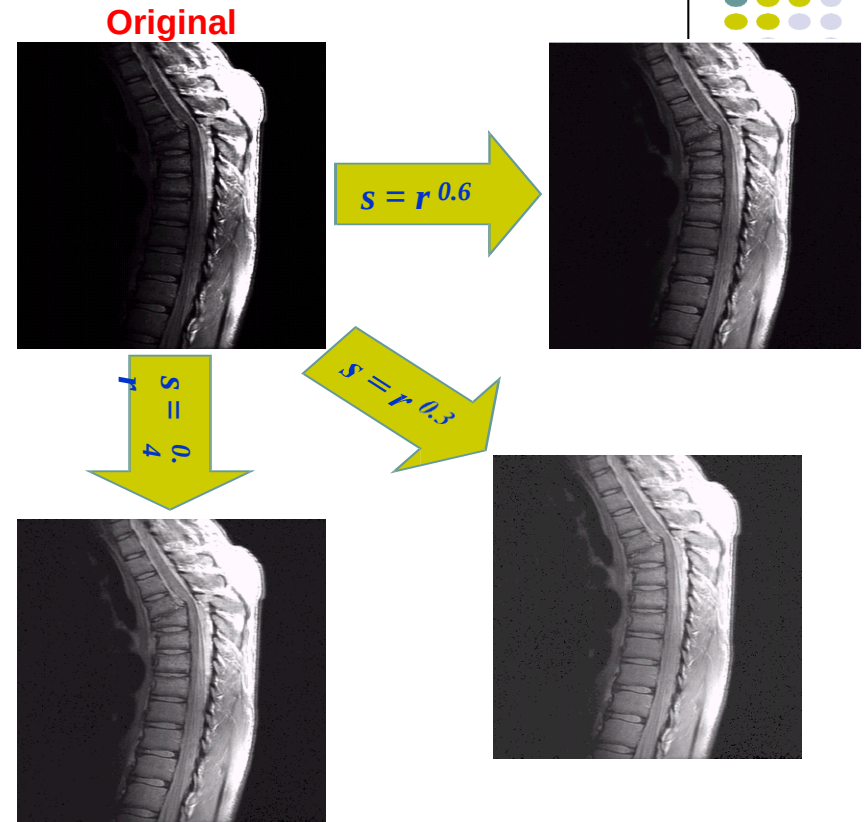
- s : New pixel value
- c : Constant
- r : Old pixel value
- γ : Power

- Map narrow range of dark input values into wider range of output values or vice versa
- Varying γ gives a whole family of curves



Power Law Example

- Magnetic Resonance (MR) image of fractured human spine
- Different power values highlight different details





Intensity Windowing

- A clamp operation, then linearly stretching image intensities to fill possible range
- To window an image in $[a,b]$ with max intensity **M**

$$f(p) = \begin{cases} 0 & \text{if } p < a \\ M \times \frac{p-a}{b-a} & \text{if } a \leq p \leq b \\ M & \text{if } p > b \end{cases}$$



Intensity Windowing

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import exposure

# Step 1: Create the matrix I (equivalent to MATLAB matrix)
I = np.array([[1, 0.9, 0.8, 0.5, 0],
              [1, 0.9, 0.8, 0.5, 0],
              [1, 0.9, 0.8, 0.5, 0],
              [1, 0.9, 0.8, 0.5, 0]])

# Step 2: Display the original matrix I
plt.imshow(I, cmap='gray')
plt.title("Original Matrix I")
plt.axis('off') # Hide the axes
plt.show()

# Step 3: Apply intensity adjustment (imadjust equivalent)
J = exposure.rescale_intensity(I, in_range=(0.5, 0.9), out_range=(0, 1))

# Step 4: Display the adjusted matrix J
plt.imshow(J, cmap='gray')
plt.title("Adjusted Matrix J")
plt.axis('off') # Hide the axes
```



Intensity Windowing

I =

1.0000	0.9000	0.8000	0.5000	0
1.0000	0.9000	0.8000	0.5000	0
1.0000	0.9000	0.8000	0.5000	0
1.0000	0.9000	0.8000	0.5000	0

J =

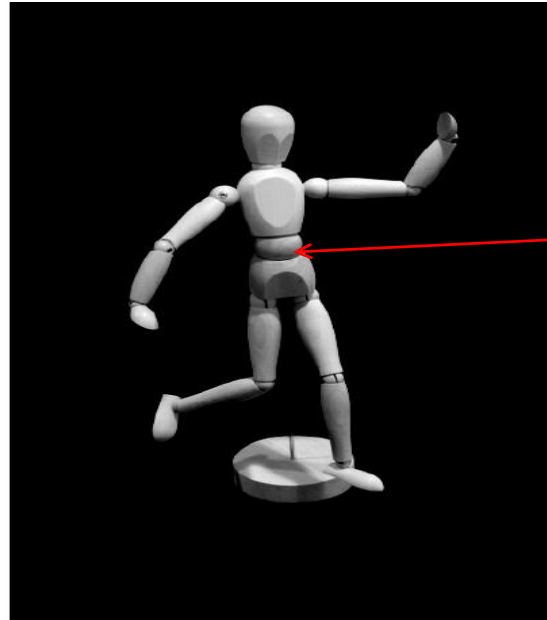
1.0000	1.0000	0.7500	0	0
1.0000	1.0000	0.7500	0	0
1.0000	1.0000	0.7500	0	0
1.0000	1.0000	0.7500	0	0



Intensity Windowing Example



Original Image



Windowed Image

Contrasts
easier to see



References

- Wilhelm Burger and Mark J. Burge, Digital Image Processing, Springer, 2008
 - Histograms (Ch 4)
 - Point operations (Ch 5)
- University of Utah, CS 4640: Image Processing Basics, Spring 2012
- Rutgers University, CS 334, Introduction to Imaging and Multimedia, Fall 2012
- Gonzales and Woods, Digital Image Processing (3rd edition), Prentice Hall